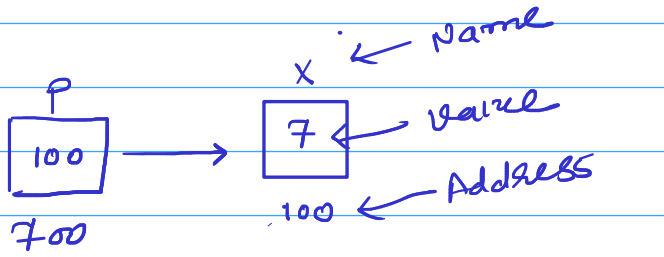
Pointers

* → value of

& → Address of



Starting of x = 100

Ending of x = 103

100 – 103

int x = 7;

① → Declaration

✓ int *p;

✓ float *P2;

✓ char *P3;

① + ②

int *p = &x; ✓

② Assignment →

p = 100; ✗

p = &x; ✓

0X1A2237

Printf("%p", p); // 100

Printf("%p", &x); // 100 ✓

③ Accessing the data

p	// 100 → Add. of x	Printf("%p", p);	// 100
*p	// 7 → value of x	Printf("%d", *p);	// 7
&p	// 700 → Add. of p	Printf("%p", &p);	// 700

1) int x=5, y=7, *p, *q;

2) p = &x; ✓

3) y = *p; ✓ // y = x;

4)

5) *p = 70; ✓

6) p = &y; ✓

7) q = &x; ✓

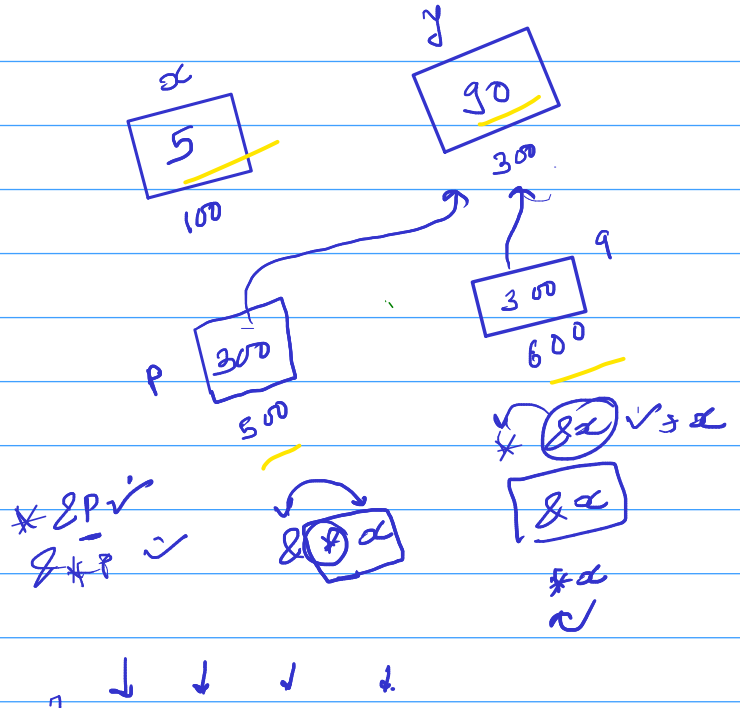
8)

9) *q = (*p); ✓

10) q = p; ✓

11)

12) *q = 90; ✓



13) printf("%d %d %d %d", x, y, *p, *q); // 5 90 90 90

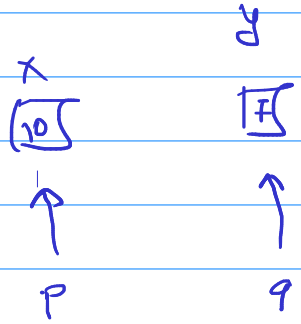
14) printf("%p %p %p %p", &x, &y, &p, &q); // 100 300 500 600

15) printf("%p %p %p", &y, p, q); // 300 300 300

*q = *p;

y = 10;

y = x;



&(*p); ⇒ p; ⇒ &y;

*&p; ⇒ p ⇒ &y;

*(350)

↓

5000

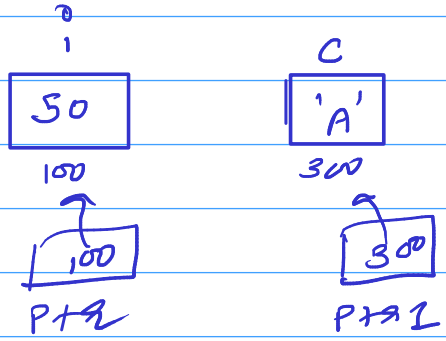
int * ptr1 ; 100

char * ptr2 ; 300

int i ; char c ;

ptr1 = &i ;

ptr2 = &c ;



ptr++ ; ✓ 100 —ptr++;→ 104 (100 + 1 = 104)

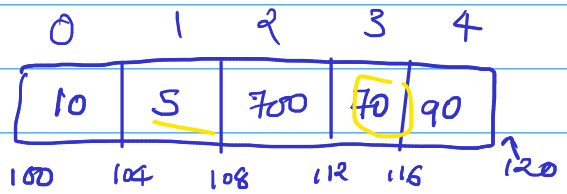
---

int arr[5] ;

→ int *ptr ;

ptr = &arr[0] ;

// ptr = arr ;



*ptr = 10 ;

ptr = 10 ; ∝

// Pointer incrementation

✓ ① ptr++ ; // 100 → 104 scale factor

ptr = arr ;

✓ ② *(ptr + 3) = 70 ; // Pointer + offset (3)

printf("%p", ptr) ; // 100

✓ ③ arr[2] = 700 ; // index

↳ *(arr + 2) = 700 ; // log

→ 2[arr] = 500 ; valid ! ✓

↳ *(2 + arr)

bracket does the work of adding the index and then derefrencing it..

→ ptr[2] = 70; ✓

↳ *(ptr+2)

→ 2[ptr] = 70; ✓

→ ptr++; *ptr → 5

→ arr++; *arr → 5 ✗ ONLY

Constantly points to base